THE NEOTIA UNIVERSITY

Jhingar Pole, Diamond Harbour Rd, Sarisha, West Bengal 743368

INTERNSHIP PROJECT REPORT

AI Resume Screening & Candidate Ranking System

Submitted in Partial Fulfillment of the Requirements for the Bachelor of Computer Applications (BCA) Programme

Group Name: A

School of Science & Technology

Bachelor of Computer Applications (BCA)

Academic Session: 2025–2027

PROJECT TEAM MEMBERS

Member 1

Name	Debasish Jana
UID	TNU20230692100063
Registration No.	TNU20230692100063
Semester	6th
Programme	Bachelor of Computer Applications (BCA)
Role	Project coordination, Data processing, Development & Testing.

Member 2

Name	Rahul Khan
UID	TNU2023069100047
Registration No.	TNU2023069100047
Semester	6th
Programme	Bachelor of Computer Applications (BCA)
Role	Data Collection, Data Preparation & Data Visualization

Member 3

Name	Soumili Mandal
UID	TNU2023069200034
Registration No.	TNU2023069200034
Semester	6th
Programme	Bachelor of Computer Applications (BCA)
Role	Presentation preparation & Documentation

ACKNOWLEDGEMENT

We express our sincere gratitude to The Neotia University for providing us the opportunity and resources to undertake this project as part of our Bachelor of Computer Applications (BCA) curriculum. The exposure and learning gained through this project have strengthened our understanding of applied machine learning and software development.

We would like to express our heartfelt appreciation to our project mentor, Riya Bera , for the continuous guidance, encouragement, and valuable feedback provided throughout the development of this project, titled “AI Resume Screening & Candidate Ranking System.”

Finally, we would like to thank everyone who directly or indirectly contributed to the successful completion of this project. This experience has strengthened our technical knowledge and inspired us to continue exploring the fields of Artificial Intelligence and Machine Learning.

Team Members, Group A

TABLE OF CONTENTS

PROJECT TEAM MEMBERS 2
ACKNOWLEDGEMENT 3
TABLE OF CONTENTS 4
Application Flow Overview 6
Phase 1: Understanding the Business Problem 6
Phase 2: Import Required Libraries 7
Phase 3: PDF Text Extraction 8
Phase 4: TF-IDF Vectorization & Cosine Similarity Ranking 9
Phase 5: Streamlit User Interface Integration 11
Phase 6: Running the Application — Input & Output Example 13
Phase 7: Results and Business Insights 15
Phase 8: Advantages and Limitations 15
Phase 9: Future Scope and Recommendations 16
Final Conclusion 17
Member Contribution 18
References 18

Application Flow Overview

Phase 1: Understanding the Business Problem

Phase 2: Import Required Libraries

Phase 3: PDF Text Extraction

Phase 4: TF-IDF Vectorization & Cosine Similarity Ranking

Phase 5: Streamlit User Interface Integration

Phase 6: Running the Application — Input & Output Example

Phase 7: Results and Business Insights

Phase 8: Advantages and Limitations

Phase 9: Future Scope and Recommendations

Final Conclusion

Member Contribution

References

Application Flow Overview

Unlike a multi-stage machine learning pipeline that saves intermediate CSV files between phases, this application runs as a single interactive Streamlit session: every step below happens in-memory, in sequence, each time a user submits a job description and resumes. The table below summarizes what flows in and out of each stage.

Library / Type	Function / Method	Definition
Input	Job description (text) + resumes (PDF)	Entered / uploaded by the user through the Streamlit interface.
Phase 3 Output	Extracted resume text (in-memory)	Plain text pulled from every page of each uploaded PDF.
Phase 4 Output	TF-IDF vectors + similarity scores (in-memory)	Numeric representation of the job description and every resume, and their cosine similarity scores.
Phase 5 Output	Ranked results table (displayed)	A pandas DataFrame of resume filenames and scores, sorted highest first, rendered on screen.

Phase 1: Understanding the Business Problem

Introduction

This phase defines the project's purpose before any coding begins. Recruiters and hiring teams routinely receive far more resumes than can be carefully read for every open role, and manually comparing each one against a job description is slow, inconsistent, and prone to reviewer bias. The goal is to automate the first pass of shortlisting.

Detailed Explanation

The core task is a text-similarity ranking problem, not a classification problem: given one job description and a set of candidate resumes, the system must

produce a ranked order of candidates by how closely each resume's content matches the job description. This framing determines every downstream decision — the choice of PDF parsing to get raw text, TF-IDF to represent that text numerically, and cosine similarity to compare it against the job description.

Conclusion from this Phase

We concluded that this is a text-similarity ranking problem: the system must score and order resumes by how closely their content matches a given job description, not classify them into fixed categories.

Library / Type	Function / Method	Definition
Concept	Text similarity ranking	Ordering documents by how closely their content matches a reference document.
Concept	Job description	The reference text the system ranks every resume against.
Concept	Business objective	Give recruiters a fast, consistent first-pass shortlist instead of manual reading.

Phase 2: Import Required Libraries

Introduction

This phase imports every external library the application depends on: one for the web interface, one for reading PDF files, one for tabular data handling, and one for the machine learning (vectorization and similarity) logic.

Detailed Explanation

Streamlit provides the interactive web UI — the title, text area, file uploader, and results display all come from this single library. PyPDF2's PdfReader is used to open and read uploaded PDF files. pandas builds and sorts the final results table. scikit-learn supplies both TfidfVectorizer, which converts text into numeric vectors, and cosine_similarity, which measures how alike two vectors are.

Conclusion from this Phase

We concluded that four libraries cover every requirement of the app: Streamlit for the interface, PyPDF2 for PDF parsing, pandas for tabular results, and scikit-learn for the ranking logic.

Input:

```
import streamlit as st
from PyPDF2 import PdfReader
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
```

Library / Type	Function / Method	Definition
Streamlit	import streamlit as st	Imports Streamlit for building the interactive web interface.
PyPDF2	from PyPDF2 import PdfReader	Imports the PDF reader used to open and extract text from resumes.
pandas	import pandas as pd	Imports pandas for building and sorting the results table.
scikit-learn	TfidfVectorizer	Converts text documents into TF-IDF weighted numeric vectors.
scikit-learn	cosine_similarity	Computes the cosine similarity between two sets of vectors.

Phase 3: PDF Text Extraction

Introduction

Uploaded resumes arrive as PDF files, which cannot be compared as text directly. This phase converts each PDF into plain text so it can be processed by the ranking logic in the next phase.

Detailed Explanation

The extract_text_from_pdf() function takes an uploaded file object, opens it with PyPDF2's PdfReader, and loops through every page in the document. For each page, page.extract_text() pulls out the readable text, which is appended to a running string. Once every page has been processed, the complete resume text is returned to the caller.

Conclusion from this Phase

We concluded that a simple page-by-page loop with PyPDF2 is sufficient to convert an uploaded PDF resume into a single plain-text string ready for vectorization.

Input:

```
def extract_text_from_pdf(file):
    pdf = PdfReader(file)
    text = ""
    for page in pdf.pages:
        text += page.extract_text()
    return text
```

Library / Type	Function / Method	Definition
PyPDF2	PdfReader(file)	Opens an uploaded PDF file object for reading.
PyPDF2	pdf.pages	An iterable of every page in the opened PDF.
PyPDF2	page.extract_text()	Extracts the readable text content from a single page.
Python	for loop	Iterates through every page of the PDF in order.
Python	+= (string concatenation)	Appends each page's text onto the running resume text.
Python	return	Returns the fully assembled resume text to the caller.

Phase 4: TF-IDF Vectorization & Cosine Similarity Ranking

Introduction

This is the core ranking phase. It converts the job description and every resume into comparable numeric vectors, then scores how similar each resume is to the job description.

Detailed Explanation

The rank_resumes() function places the job description at index 0 of a combined documents list, followed by every resume's text. A single TfidfVectorizer is fit across this entire list so that every document shares the same vocabulary and vector space — this matters because TF-IDF weights are only comparable when computed together. TF-IDF assigns higher weight to words that are frequent in a given document but rare across the whole document set, so common filler words contribute little while distinctive skills and keywords (e.g. “scikit-learn”, “Flask”) carry more weight. Once every document has a vector, the job description's vector is separated from the resume vectors, and cosine_similarity computes the angle-based similarity between the job description and every resume in a single call. The result is flattened into a simple one-dimensional array of scores, one per resume, in the same order they were uploaded.

Conclusion from this Phase

We concluded that fitting one shared TF-IDF vectorizer across the job description and all resumes, then scoring cosine similarity against the job description vector, produces a reliable relative ranking of candidates without any manual keyword rules.

Input:

```
def rank_resumes(job_description, resumes):
    # Combine job description with resumes
    documents = [job_description] + resumes
    vectorizer = TfidfVectorizer().fit_transform(documents)
    vectors = vectorizer.toarray()
    # Calculate cosine similarity
    job_description_vector = vectors[0]
    resume_vectors = vectors[1:]
    cosine_similarities = cosine_similarity(
```

```
        [job_description_vector],    resume_vectors
    ).flatten()
    return cosine_similarities
```

Library / Type	Function / Method	Definition
Python	[job_description] + resumes	Combines the job description and all resumes into one ordered document list.
scikit-learn	TfidfVectorizer()	Creates a TF-IDF vectorizer instance with default settings.
scikit-learn	fit_transform(documents)	Learns the shared vocabulary and transforms every document into a TF-IDF vector.
NumPy (via sklearn)	toarray()	Converts the sparse TF-IDF matrix into a dense NumPy array.
Python	vectors[0] / vectors[1:]	Slices the array to separate the job description vector from the resume vectors.
scikit-learn	cosine_similarity()	Computes the cosine similarity between the job description vector and every resume vector.
NumPy	flatten()	Converts the 2-D similarity result into a simple 1-D array of scores.

Phase 5: Streamlit User Interface Integration

Introduction

This phase wires the two functions above into an interactive web app: it collects the job description and resumes from the user, calls the ranking logic, and displays the results.

Detailed Explanation

The script sets the page title with st.title(), then renders a text area for the job description and a multi-file uploader restricted to PDF files. Once both a job

description and at least one resume are present, the app extracts text from every uploaded resume using extract_text_from_pdf(), calls rank_resumes() to score them, and builds a pandas DataFrame pairing each resume's filename with its score. The DataFrame is sorted by score in descending order and displayed with st.write(), so the best-matching candidate always appears first.

Conclusion from this Phase

We concluded that a linear Streamlit script — input widgets, a conditional ranking step, and a sorted results table — is sufficient to turn the ranking logic into a usable interactive tool.

Input:

```
st.title("AI Resume Screening & Candidate Ranking System")

st.header("Job Description")
job_description = st.text_area("Enter the job description")

st.header("Upload Resumes")
uploaded_files = st.file_uploader("Upload PDF files", type=["pdf"], accept_multiple_files=True)

if uploaded_files and job_description:
    st.header("Ranking Resumes")
    resumes = []
    for file in uploaded_files:
        text = extract_text_from_pdf(file)
        resumes.append(text)

    scores = rank_resumes(job_description, resumes)

    results = pd.DataFrame({
        "Resume": [file.name for file in uploaded_files],
        "Score": scores
    })
    results = results.sort_values(by="Score",
```

```
ascending=False)
    st.write(results)
```

Library / Type	Function / Method	Definition
Streamlit	st.title()	Displays the application's main title.
Streamlit	st.header()	Displays section headers such as “Job Description” and “Upload Resumes”.
Streamlit	st.text_area()	Renders a multi-line text box for entering the job description.
Streamlit	st.file_uploader()	Renders an upload widget accepting multiple PDF resumes.
Streamlit	st.write()	Displays the final ranked results table on the page.
pandas	pd.DataFrame()	Builds a table pairing each resume's filename with its similarity score.
pandas	sort_values()	Sorts the results table by score, highest match first.
Python	if condition	Runs the ranking logic only once a job description and resumes are both present.
Python	for loop	Extracts text from every uploaded resume file in turn.

Phase 6: Running the Application — Input & Output Example

Introduction

This phase demonstrates the application end-to-end using the actual ranking function, so the input/output behaviour shown here is computed, not illustrative.

Sample Input — Job Description

```
We are looking for a Python Developer with strong
experience                    in                    data
analysis and machine learning. The candidate should
```

```
be skilled in Python, pandas, scikit-learn, SQL, and have experience building REST APIs with Flask or Django. Familiarity with data visualization and version control using Git is required. Good communication and problem-solving skills are essential.
```

Sample Input — Uploaded Resumes

- Resume_Aisha_Khan.pdf — Python developer, data analysis, scikit-learn, SQL, Flask, Git, matplotlib.
- Resume_Karan_Verma.pdf — Data scientist, Python, pandas, scikit-learn, SQL, Flask, Git.
- Resume_Rohan_Das.pdf — Frontend developer, HTML/CSS/JavaScript/React, no Python or data analysis experience.
- Resume_Meera_Iyer.pdf — Java/Spring Boot backend engineer, basic SQL, limited Python/ML exposure.

Output — Ranked Results

Rank	Resume	Similarity Score
1	Resume_Aisha_Khan.pdf	0.5660
2	Resume_Karan_Verma.pdf	0.4682
3	Resume_Rohan_Das.pdf	0.2907
4	Resume_Meera_Iyer.pdf	0.1959

Conclusion from this Phase

We concluded that the two resumes sharing direct Python, data-analysis, and Flask/SQL vocabulary with the job description (Aisha Khan and Karan Verma) score highest, while the frontend-focused and Java-focused resumes score progressively lower, confirming the ranking behaves as intended.

Phase 7: Results and Business Insights

Detailed Explanation

The ranking behaviour observed in Phase 6 reflects TF-IDF's core property: resumes that genuinely share technical vocabulary with the job description score higher, without any hand-coded keyword rules. This has direct, practical value for a recruiter screening a large applicant pool.

Conclusion from this Phase

We concluded that an automated TF-IDF + cosine similarity ranking gives recruiters a consistent, explainable first-pass shortlist that highlights the most textually relevant candidates first.

Library / Type	Function / Method	Definition
Business Analysis	Relative ranking	Orders every uploaded resume from strongest to weakest match for a specific role.
Business Analysis	Consistency	The same job description and resumes always produce the same ranking, removing reviewer-to-reviewer variance.
Business Analysis	Time savings	Recruiters can focus manual review on the top-ranked candidates instead of reading every resume.

Phase 8: Advantages and Limitations

Advantages

- Removes the need to manually read every resume for an initial shortlist.
- Produces consistent, repeatable rankings — the same inputs always give the same output.
- Simple, lightweight approach with no external paid API or service dependency.
- Interactive web interface requires no coding knowledge to operate.

Limitations

- TF-IDF is a keyword/vocabulary-overlap measure — it does not understand meaning, so different wording for an equivalent skill may score lower than it should.
- PDF text extraction quality depends on how the original PDF was created; scanned/image-based resumes without a text layer will not extract usefully.
- The ranking is a screening aid, not a hiring decision — it does not verify claims made in a resume.

Conclusion from this Phase

We concluded that the system's simplicity is both its main strength and its main constraint: it is easy to run and reason about, but it ranks on vocabulary overlap rather than true semantic meaning.

Phase 9: Future Scope and Recommendations

- Replace or augment TF-IDF with semantic embeddings (e.g. sentence-transformers) to capture meaning rather than exact wording.
- Add a backend API (e.g. Flask/FastAPI) so the ranking engine can be used outside the Streamlit UI.
- Add authentication and persistent storage so recruiters can save and revisit past screening sessions.
- Highlight the specific matched keywords/skills for each resume alongside its score.
- Use the tool as an early-stage support aid for recruiters, not a replacement for human judgement in hiring decisions.

Conclusion from this Phase

We concluded that semantic matching, a standalone API, and persistence are the highest-value next steps to evolve this project from a classroom prototype into a production-ready screening tool.

Final Conclusion

This project successfully implements an end-to-end AI-assisted resume screening workflow. Starting from the business problem of slow, inconsistent manual resume review, the project built a working pipeline: PDF resumes are converted to text, the job description and every resume are represented as TF-IDF vectors in a shared vocabulary, and cosine similarity ranks every candidate by relevance against the job description.

The Phase 6 demonstration confirmed the ranking behaves sensibly on realistic sample data: candidates whose resumes shared genuine technical overlap with the job description were ranked above those that did not, without any hand-coded rules. Wrapped in an accessible Streamlit interface, the result is a lightweight tool that gives recruiters a consistent, explainable first-pass shortlist.

The project also surfaced clear next steps — semantic embeddings, a standalone API, and persistent storage — that would extend this prototype toward a more capable, production-ready applicant screening system. As with any automated screening aid, the tool is intended to support recruiter judgement, not replace it.

Member Contribution

The table below summarizes which part of the project each team member was responsible for.

Member Name	UID	Contribution
Debasish Jana	TNU2023069100063	PDF text extraction module, requirements setup
Rahul Khan	TNU2023069100047	TF-IDF & cosine similarity ranking logic
Soumili Mandal	TNU2023069200034	Streamlit UI design & integration

References

- Streamlit Documentation — https://docs.streamlit.io
- PyPDF2 Documentation — https://pypdf2.readthedocs.io
- scikit-learn TfidfVectorizer Documentation — https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html
- scikit-learn cosine_similarity Documentation — https://scikit-learn.org/stable/modules/generated/sklearn.metrics.pairwise.cosine_similarity.html